OpenCell Design: An Open-Source Python Library for Construction, Energetic and Technoeconomic Modeling of Battery Cell Designs

Nicholas Siemons, Adrian Yao, Will Chueh

3 July 2026

Summary

OpenCell Design is an open-source Python library that provides a hierarchical, composable API for building virtual battery cells from individual components and materials. The software implements a modeling hierarchy that mirrors the physical construction of a cell — spanning materials, formulations, electrodes, layups, electrode assemblies, encapsulations, and complete cells. Users can construct cylindrical, prismatic, and pouch cell architectures using wound jelly rolls, stacked assemblies, or z-fold stacked assemblies.

OpenCell Design calculates mass and cost at every level of the hierarchy, enabling direct assessment of economic viability alongside electrochemical performance. A propagation system automatically updates all dependent calculations when any parameter changes, eliminating the need to manually rebuild cell models. Interactive browser-based visualizations built on Plotly (Plotly Technologies Inc. 2015) provide immediate feedback through cross-section views, voltage–capacity curves, and hierarchical cost and mass breakdowns.

Statement of Need

The global transition to electrified transportation and grid-scale energy storage has placed increasing demands on battery technology (Chu and Majumdar 2012; Dunn et al. 2011). Metal-ion batteries are the dominant technology for applications from portable electronics to electric vehicles (Goodenough and Park 2013; Blomgren 2017), and both thermodynamic performance and technoeconomic metrics are critical to their continued development (Nykvist and Nilsson 2015; Ziegler and Trancik 2021). Understanding the design and material drivers behind cell performance and cost requires modeling at multiple scales — a single cell comprises dozens of interacting parameters across materials, formulations, current collectors, separators, electrodes, and encapsulations.

Several tools address aspects of this challenge. PyBaMM (Sulzer et al. 2021) provides physics-based electrochemical simulation but does not model the geometric construction of cells or calculate mass and cost breakdowns. BatPaC (Nelson et al. 2019) and CAMS (The Faraday Institution 2024) estimate manufacturing cost and performance but are implemented as Excel workbooks, limiting extensibility and programmatic integration. Both are also unidirectional models — outputs depend on a fixed set of inputs, and bidirectional parameter setting is not possible. Other tools such as electrode formulation calculators address isolated aspects of cell design rather than the complete hierarchy from materials to cells.

OpenCell Design fills this gap by providing an open-source, programmatic tool that combines hierarchical cell construction with integrated cost and performance calculations in an extensible, composable framework.

Software Architecture

The software is distributed as three complementary Python packages, each with its own repository, test suite, and documentation:

- **steer-core** (github.com/stanford-developers/steer-core) provides foundational utilities shared across the platform, including validation, serialization with LZ4 compression, bidirectional property propagation, type checking, and interactive Plotly-based plotting mixins.
- **steer-materials** (github.com/stanford-developers/steer-materials) defines the material layer — metals, solvents, and volumed material mixins that track density, cost, mass, and volume with automatic unit conversion and range validation.
- **steer-opencell-design** (github.com/stanford-developers/steer-opencell-design) is the primary package described in this paper. It builds on the previous two to implement the full cell-modeling hierarchy from active materials through to complete cells.

This separation of concerns allows each package to be developed, tested, and versioned independently while ensuring that common behaviors — such as serialization, validation, and change propagation — are defined once in `steer-core` and inherited throughout the stack.

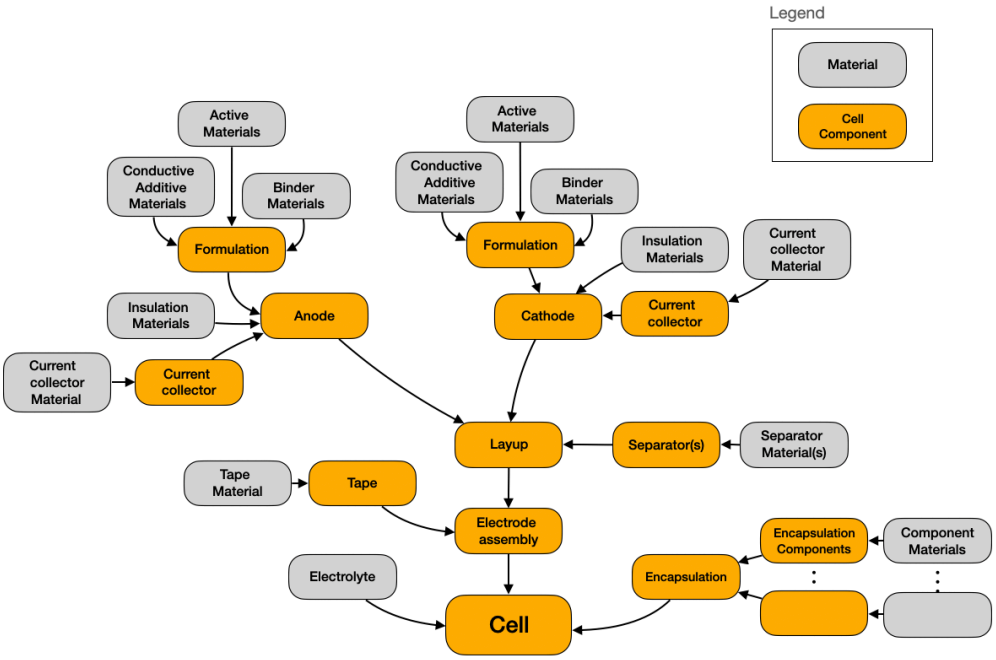

Figure 1: The OpenCell Design modeling hierarchy. Components at each level compose into the next, from materials through to complete cells.

Key Features

Hierarchical modeling architecture. Components are realized as Python classes organized into a hierarchy that mirrors the physical construction of a cell. Materials are composed into formulations, formulations are coated onto current collectors to form electrodes, electrodes are arranged into layups, layups are wound or stacked into electrode assemblies, and assemblies are encapsulated into complete cells. Properties are calculated at the lowest possible level — specific capacity in the active material, areal capacity in the electrode, total capacity in the cell.

Bidirectional parameter propagation. Each object maintains a reference to its parent. When a user modifies a parameter and calls `propagate_changes()`, the system re-assigns the modified object to its

parent's setter, triggering recalculation at each level up to the cell. A finer-grained `update()` method propagates changes by a single level, enabling complex studies such as comparing designs at constant volume or constant N/P ratio.

Extensible object-oriented design. Base classes implemented as abstract base classes define the interface for each component type. New materials, current collector geometries, and entirely new cell formats can be added by subclassing. To demonstrate this, OpenCell Design includes a flex-frame cell class — a novel solid-state cell architecture — defined in approximately 1,000 lines of code by inheriting the existing base classes and composing existing components.

Serialization and visualization. A compact binary serialization format allows cell designs to be saved, shared, and version-controlled. Interactive Plotly-based visualizations provide cross-section views, top-down views, voltage–capacity plots, and sunburst cost/mass breakdowns at any level of the hierarchy.

Numerical methods. The library uses NumPy (Harris et al. 2020) and SciPy (Virtanen et al. 2020) for calculations, including closed-form and adaptive Runge–Kutta integration of thickness-dependent jelly roll spirals accelerated with Numba (Lam et al. 2015), and Brent's root-finding algorithm (Brent 1971) for constraint satisfaction.

Quality Control

OpenCell Design includes a comprehensive test suite comprising 21 test modules with over 900 individual test cases, organized to mirror the package structure. Tests cover materials, formulations, electrodes, current collectors, separators, layups, electrode assemblies, containers, and complete cells.

Acknowledgements

We thank the members of the STEER group at Stanford University for their feedback during the development of this software.

References

Blomgren, George E. 2017. "The Development and Future of Lithium Ion Batteries." *Journal of The Electrochemical Society* 164 (1): A5019–25. https://doi.org/10.1149/2.0251701jes.

Brent, Richard P. 1971. "An Algorithm with Guaranteed Convergence for Finding a Zero of a Function." *The Computer Journal* 14 (4): 422–25. https://doi.org/10.1093/comjnl/14.4.422.

Chu, Steven, and Arun Majumdar. 2012. "Opportunities and Challenges for a Sustainable Energy Future." *Nature* 488 (7411): 294–303. https://doi.org/10.1038/nature11475.

Dunn, Bruce, Haresh Kamath, and Jean-Marie Tarascon. 2011. "Electrical Energy Storage for the Grid: A Battery of Choices." *Science* 334 (6058): 928–35. https://doi.org/10.1126/science.1212741.

Goodenough, John B, and Kyu-Sung Park. 2013. "The Li-Ion Rechargeable Battery: A Perspective." *Journal of the American Chemical Society* 135 (4): 1167–76. https://doi.org/10.1021/ja3091438.

Harris, Charles R, K Jarrod Millman, Stéfan J van der Walt, et al. 2020. "Array Programming with NumPy." *Nature* 585 (7825): 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Lam, Siu Kwan, Antoine Pitrou, and Stanley Seibert. 2015. "Numba: A LLVM-Based Python JIT Compiler." *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6. https://doi.org/10.1145/2833157.2833162.

Nelson, Paul A, Shabbir Ahmed, Kevin G Gallagher, and Dennis W Dees. 2019. *Modeling the Performance and Cost of Lithium-Ion Batteries for Electric-Drive Vehicles, Third Edition*. ANL/CSE-19/2. Argonne National Laboratory. https://doi.org/10.2172/1503280.

Nykvist, Björn, and Måns Nilsson. 2015. "Rapidly Falling Costs of Battery Packs for Electric Vehicles." *Nature Climate Change* 5 (4): 329–32. https://doi.org/10.1038/nclimate2564.

Plotly Technologies Inc. 2015. *Plotly: Collaborative data science*. Https://plot.ly.

Sulzer, Valentin, Scott G Marquis, Robert Timms, Martin Robinson, and S Jon Chapman. 2021. "Python Battery Mathematical Modelling (PyBaMM)." *Journal of Open Research Software* 9 (1): 14. https://doi.org/10.5334/jors.309.

The Faraday Institution. 2024. *Cell Analysis and Modelling System (CAMS)*. Https://www.faraday.ac.uk/fi-cell-modelling-workbook/.

Virtanen, Pauli, Ralf Gommers, Travis E Oliphant, et al. 2020. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods* 17: 261–72. https://doi.org/10.1038/s41592-019-0686-2.

Ziegler, Micah S, and Jessika E Trancik. 2021. "Re-Examining Rates of Lithium-Ion Battery Technology Improvement and Cost Decline." *Energy & Environmental Science* 14 (4): 1635–51. https://doi.org/10.1039/D0EE02681F.